

**VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING**



Microprocessors – Microcontrollers
Laboratory Report 2
Timer Interrupt and LED Scanning

Advisor(s): Phan Văn Sỹ
Student(s): Vũ Thiện Nhân - 2452888

HO CHI MINH CITY, OCTOBER 2026

Table of Contents

Table of Contents.....	2
1. Introduction.....	3
2. Hardware Design	3
2.1 Shared Schematic (Exercises 1-10)	3
3. Exercise 1: Two-LED Alternating Scan	4
4. Exercise 2: Four-LED Scan with DOT Blink	4
5. Exercise 3: update7SEG(int index) Function	5
6. Exercise 4: 1Hz Scanning Frequency	6
7. Exercise 5: Digital Clock - updateClockBuffer()	7
8. Exercise 6: Software Timer	7
9. Exercise 7: Software-Timer-Driven Clock	9
10. Exercise 8: Fully Non-Blocking Main Loop	9
11. Exercise 9: updateLEDMatrix(int index) Function	11
12. Exercise 10: Full Analog Matrix Animation	12

1. Introduction

This report presents the implementation of Laboratory 2 (Timer Interrupt and LED Scanning) of the Microprocessors - Microcontrollers course, using the STM32F103C6 microcontroller simulated in Proteus 8 Professional and programmed with STM32CubeIDE (HAL library).

The TIM2 configuration is shared across all ten exercises: Clock Source = Internal Clock (HSI 8MHz), Prescaler = 7999, Counter Period (ARR) = 9. $8\text{MHz} / (7999+1) = 1000\text{Hz}$, so the counting period is 1ms. An interrupt is raised every time the TIM2 counter counts from 0 to 9 (10 ticks), so the interrupt frequency is $1000\text{Hz} / 10 = 100\text{Hz}$, i.e. an interrupt period of $1/100\text{Hz} = 10\text{ms}$.

2. Hardware Design

2.1 Shared Schematic (Exercises 1-10)

All ten exercises share the same Proteus schematic, each simulated as its own CubeIDE project (Debug is pointed at that exercise's own .hex file). The circuit includes 4x common-anode 7-segment displays driven through PNP transistors Q1-Q4 (EN0-EN3), the heartbeat LED D1 and the DOT LEDs D2/D3 (PA4), and the 8x8 LED matrix driven through the ULN2803 NPN transistor array (ENM0-ENM7 column enables, ROW0-ROW7 row data).

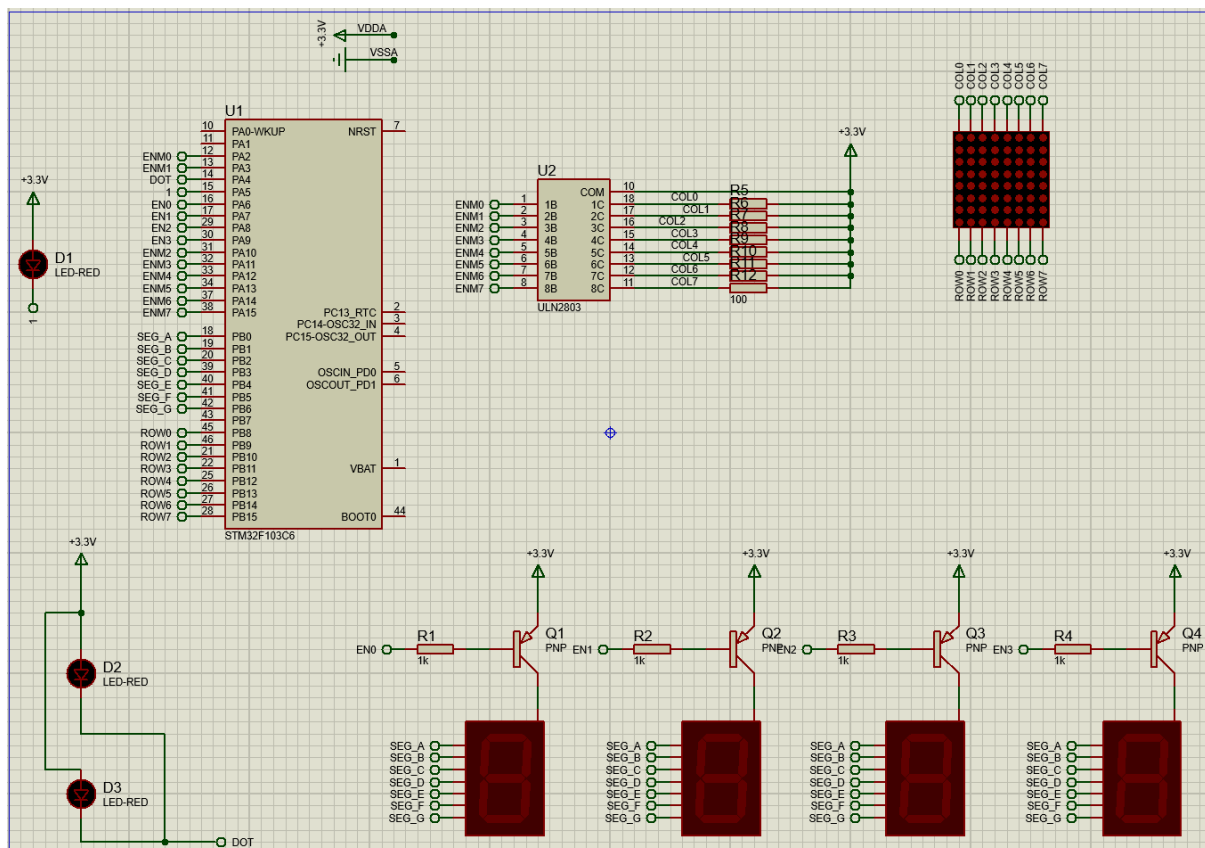


Figure 2.1: Proteus schematic shared across Exercises 1-10

[mcu-mpu_lab_schematic/lab2_ex1-10.png](#) at [main · garrettnhan-coder/mcu-mpu_lab_schematic](#)

3. Exercise 1: Two-LED Alternating Scan

Interface with 2 common-anode 7-segment LEDs through PNP transistors (EN0, EN1 switch Q1, Q2 on/off) and 7 shared segment signal lines (SEG_A - SEG_G). The display shows "1" on the first LED and "2" on the second, switching between them every 500ms (scanning frequency = $1/(2 \times 0.5s) = 1Hz$).

Report 1: Capture your schematic from Proteus and show it in the report (see Figure 2.1).

Report 2: Present your source code in the HAL_TIM_PeriodElapsedCallback function.

Listing 3.1 — Exercise 1 - HAL_TIM_PeriodElapsedCallback

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM2)
    {
        led_counter--;
        if (led_counter <= 0)
        {
            led_counter = 50;
            HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);
        }

        counter--;
        if (counter <= 0)
        {
            counter = 50;
            if (status == 0)
            {
                // LED 2 on
                HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, SET);
                HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, RESET);
                display7SEG(2);
                status = 1;
            } else
            {
                // LED 1 on
                HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, SET);
                HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, RESET);
                display7SEG(1);
                status = 0;
            }
        }
    }
}
```

Short question: What is the frequency of the scanning process?

Each LED is turned on for 500ms, and there are 2 LEDs, so the full scanning period $T = 2 \times 500ms = 1000ms = 1s$. Therefore the scanning frequency $f = 1/T = 1Hz$.

4. Exercise 2: Four-LED Scan with DOT Blink

Extend the system to 4 seven-segment LEDs and add two DOT LEDs (connected to PA4) in the middle. The two DOT LEDs blink every second, digit "3" is shown on the third LED and "0" on the fourth (representing 12 and a half o'clock). The switching time between 7-segment LEDs is still 500ms, implemented inside the timer interrupt function.

Report 1: Capture your schematic from Proteus and show it in the report (see Figure 2.1).

Report 2: Present your source code in the HAL_TIM_PeriodElapsedCallback function.*Listing 4.1 — Exercise 2 - HAL_TIM_PeriodElapsedCallback*

```

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM2) {
        // blink D1 (LED_RED, PA5)
        led_counter--;
        if (led_counter <= 0) {
            led_counter = 50;
            HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);
        }

        // blink DOT
        dot_counter--;
        if (dot_counter <= 0) {
            dot_counter = 100;
            HAL_GPIO_TogglePin(DOT_GPIO_Port, DOT_Pin);
        }

        // scan LED
        counter--;
        if (counter <= 0) {
            counter = 50;
            switch (status) {
                case 0:
                    enableLed(0);
                    display7SEG(1);
                    status = 1;
                    break;
                case 1:
                    enableLed(1);
                    display7SEG(2);
                    status = 2;
                    break;
                case 2:
                    enableLed(2);
                    display7SEG(3);
                    status = 3;
                    break;
                case 3:
                    enableLed(3);
                    display7SEG(0);
                    status = 0;
                    break;
                default:
                    break;
            }
        }
    }
}

```

Short question: What is the frequency of the scanning process?

The system now has 4 seven-segment LEDs, each turned on for 500ms, so the full scanning period $T = 4 \times 500\text{ms} = 2000\text{ms} = 2\text{s}$. Therefore the scanning frequency $f = 1/T = 0.5\text{Hz}$.

5. Exercise 3: update7SEG(int index) Function

Implement the function `update7SEG(int index)` to decouple the data to be displayed (the `led_buffer[4]` array) from the LED-scanning logic, making the program cleaner and easier to extend in the following exercises.

Report 1: Present the source code of the update7SEG function.*Listing 5.1 — Exercise 3 - update7SEG function*

```

void update7SEG(int index)
{
    switch (index) {
        case 0:
            enableLed(0);
            display7SEG(led_buffer[0]);
            break;
        case 1:
            enableLed(1);
            display7SEG(led_buffer[1]);
            break;
        case 2:
            enableLed(2);
            display7SEG(led_buffer[2]);
            break;
        case 3:
            enableLed(3);
            display7SEG(led_buffer[3]);
            break;
        default:
            break;
    }
}

```

Report 2: Present your source code in the HAL_TIM_PeriodElapsedCallback function.*Listing 5.2 — Exercise 3 - HAL_TIM_PeriodElapsedCallback*

```

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM2) {

        // blink D1 (LED_RED, PA5)
        led_counter--;
        if (led_counter <= 0) {
            led_counter = 50;
            HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);
        }

        // blink DOT every 1 sec
        dot_counter--;
        if (dot_counter <= 0) {
            dot_counter = 100;
            HAL_GPIO_TogglePin(DOT_GPIO_Port, DOT_Pin);
        }

        // scan 7 led segment, every led 500ms
        counter--;
        if (counter <= 0) {
            counter = 50;
            update7SEG(index_led++);
            if (index_led >= MAX_LED) {
                index_led = 0;
            }
        }
    }
}

```

6. Exercise 4: 1Hz Scanning Frequency

Change the period of invoking the update7SEG function so that the scanning frequency of the 4 seven-segment LEDs becomes 1Hz (each LED is turned on for 250ms, since $4 \times 250\text{ms} = 1\text{s}$). The DOT LEDs keep blinking every second as before.

Report 1: Present the source code in the HAL_TIM_PeriodElapsedCallback function.

Listing 6.1 — Exercise 4 - HAL_TIM_PeriodElapsedCallback

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM2) {

        // blink D1 (LED_RED, PA5)
        led_counter--;
        if (led_counter <= 0) {
            led_counter = 50;
            HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);
        }

        // blink DOT every 1 sec
        dot_counter--;
        if (dot_counter <= 0) {
            dot_counter = 100;
            HAL_GPIO_TogglePin(DOT_GPIO_Port, DOT_Pin);
        }

        // scan 7 led segment, every led 250ms (frequency 1Hz)
        counter--;
        if (counter <= 0) {
            counter = 25;
            update7SEG(index_led++);
            if (index_led >= MAX_LED) {
                index_led = 0;
            }
        }
    }
}
```

7. Exercise 5: Digital Clock - updateClockBuffer()

Implement a digital clock that displays hour and minute information on 2 seven-segment LEDs, through the function updateClockBuffer(), which converts the hour/minute values into 4 individual digits stored in led_buffer.

Report 1: Present the source code in the updateClockBuffer function.

Listing 7.1 — Exercise 5 - updateClockBuffer function

```
void updateClockBuffer(void)
{
    led_buffer[0] = hour / 10;
    led_buffer[1] = hour % 10;
    led_buffer[2] = minute / 10;
    led_buffer[3] = minute % 10;
}
```

8. Exercise 6: Software Timer

Implement a software timer (a software counter that is decremented every 10ms inside the TIM2 interrupt) to remove HAL_Delay() from the main loop, keeping the program non-

blocking and improving real-time behaviour. The reference program (Program 1.8 from the lab manual) is as follows:

Listing 8.1 — Reference software-timer program

```
int timer0_counter = 0;
int timer0_flag = 0;
int TIMER_CYCLE = 10;    // 10ms per timer interrupt (TIM2)

void setTimer0(int duration)
{
    timer0_counter = duration / TIMER_CYCLE;
    timer0_flag = 0;
}

void timer_run(void)
{
    if (timer0_counter > 0) {
        timer0_counter--;
        if (timer0_counter == 0) timer0_flag = 1;
    }
}

/* ---- inside main() ---- */
setTimer0(1000);           // <-- line 1
while (1) {
    if (timer0_flag == 1) {
        HAL_GPIO_TogglePin(LED_RED_GPIO_Port, LED_RED_Pin);
        setTimer0(2000);
    }
}
```

Report 1: If line 1 (`setTimer0(1000);`) is missing, what happens afterwards and why?

If the `setTimer0(1000)` call is missing, the variable `timer0_counter` keeps its default initial value of 0. Inside `timer_run()`, the condition "`if (timer0_counter > 0)`" is always false (since counter is 0 from the start), so the counter is never decremented and `timer0_flag` is never set to 1. As a result, the condition "`if (timer0_flag == 1)`" inside the `while(1)` loop never becomes true, so the LED never blinks.

Report 2: If line 1 is changed to `setTimer0(1)`, what happens afterwards and why?

With `TIMER_CYCLE = 10` (ms), `timer0_counter` is computed using integer division: $\text{duration} / \text{TIMER_CYCLE} = 1 / 10 = 0$ (rounded down). So `timer0_counter` is still 0, exactly as in the "missing line" case in Report 1. `timer_run()` never decrements the counter and `timer0_flag` is never set, so the LED still does not blink.

Report 3: If line 1 is changed to `setTimer0(10)`, what changes compared to the first two questions and why?

With `setTimer0(10)`, `timer0_counter = 10 / 10 = 1`, which is non-zero. This is the key difference: since `counter > 0`, the condition inside `timer_run()` is satisfied, and on the very next TIM2 interrupt (10ms later) the counter is decremented from 1 to 0 and `timer0_flag` is set to 1. The `while(1)` loop then detects `flag == 1`, toggles the LED, and calls `setTimer0(2000)` again to schedule the next toggle. So, unlike the first two cases (where the LED never lights up because the counter stays at 0 forever), here the LED blinks normally (just with the first toggle occurring after 10ms instead of the intended 1000ms), because 10ms is an exact multiple of `TIMER_CYCLE` (10ms), so the integer division does not lose any information.

9. Exercise 7: Software-Timer-Driven Clock

Upgrade the clock from Exercise 5 to use the software timer (timer0) for updating hour/minute/second and toggling the DOT LEDs, completely removing HAL_Delay() from the main function. The update logic is moved into the while(1) loop and only runs when timer0_flag == 1.

Report 1: Present your source code in the while loop of the main function.

Listing 9.1 — Exercise 7 - main loop

```
while (1)
{
    /* USER CODE END WHILE */
    if (timer0_flag == 1) {
        HAL_GPIO_TogglePin(DOT_GPIO_Port, DOT_Pin);

        second++;
        if (second >= 60) {
            second = 0;
            minute++;
        }
        if (minute >= 60) {
            minute = 0;
            hour++;
        }
        if (hour >= 24) {
            hour = 0;
        }
        updateClockBuffer();

        setTimer0(1000);
    }
    /* USER CODE BEGIN 3 */
}
```

10. Exercise 8: Fully Non-Blocking Main Loop

Move the update7SEG() function out of the interrupt handler and into the main function as well. At this point, HAL_TIM_PeriodElapsedCallback() only calls the timerX_run() functions to update the software counters, minimizing and shortening the processing time inside the ISR. The system now uses 3 software timers: timer0 (clock + DOT, 1000ms period), timer1 (7-segment scanning, 250ms period) and timer2 (D1 blink, 500ms period).

Report 1: Present your source code in the main function (the extra variables/functions for the 2nd and 3rd software timers are included below).

Listing 10.1 — Exercise 8 - software timer declarations

```
/* USER CODE BEGIN 0 */
int timer0_counter = 0;
int timer0_flag = 0;
int TIMER_CYCLE = 10;

void setTimer0(int duration)
{
    timer0_counter = duration / TIMER_CYCLE;
    timer0_flag = 0;
}
```

```

void timer_run(void)
{
    if (timer0_counter > 0) {
        timer0_counter--;
        if (timer0_counter == 0) timer0_flag = 1;
    }
}

// 2nd software timer, used to scan the 7-segment LEDs
int timer1_counter = 0;
int timer1_flag = 0;

void setTimer1(int duration)
{
    timer1_counter = duration / TIMER_CYCLE;
    timer1_flag = 0;
}

void timer1_run(void)
{
    if (timer1_counter > 0) {
        timer1_counter--;
        if (timer1_counter == 0) timer1_flag = 1;
    }
}

// 3rd software timer, used for D1 (LED_RED, PA5)
int timer2_counter = 0;
int timer2_flag = 0;

void setTimer2(int duration)
{
    timer2_counter = duration / TIMER_CYCLE;
    timer2_flag = 0;
}

void timer2_run(void)
{
    if (timer2_counter > 0) {
        timer2_counter--;
        if (timer2_counter == 0) timer2_flag = 1;
    }
}
/* USER CODE END 0 */

```

Listing 10.2 — Exercise 8 - main function and ISR

```

int main(void)
{
    HAL_Init();
    SystemClock_Config();

    MX_TIM2_Init();
    MX_GPIO_Init();

    /* USER CODE BEGIN 2 */
    HAL_TIM_Base_Start_IT(&htim2);
    HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin, RESET);
    updateClockBuffer();
    update7SEG(0);
    index_led = 1;
    HAL_GPIO_WritePin(DOT_GPIO_Port, DOT_Pin, SET);
    /* USER CODE END 2 */
}

```

```

/* USER CODE BEGIN WHILE */
setTimer0(1000);
setTimer1(250);
setTimer2(500);
while (1)
{
    /* USER CODE END WHILE */
    if (timer0_flag == 1) {
        HAL_GPIO_TogglePin(DOT_GPIO_Port, DOT_Pin);

        second++;
        if (second >= 60) { second = 0; minute++; }
        if (minute >= 60) { minute = 0; hour++; }
        if (hour >= 24) { hour = 0; }
        updateClockBuffer();

        setTimer0(1000);
    }

    if (timer1_flag == 1) {
        update7SEG(index_led++);
        if (index_led >= MAX_LED) { index_led = 0; }
        setTimer1(250);
    }

    if (timer2_flag == 1) {
        HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);
        setTimer2(500);
    }
    /* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM2) {
        timer_run();
        timer1_run();
        timer2_run();
    }
}
}

```

11. Exercise 9: updateLEDMatrix(int index) Function

Add an 8x8 LED matrix (MATRIX-8X8-RED) to the system, driven through the ULN2803 IC (an NPN transistor array acting as a current sink to enable/disable each column). The ENM0-ENM7 pins (PA0-PA3, PA10-PA12, PA15) control which column is enabled (SET = on, the opposite polarity of the PNP-driven EN0-EN3 logic), while the ROW0-ROW7 pins (PB8-PB15) carry the row data, written bit by bit from the corresponding byte in the `matrix_buffer[8]` array.

Report 1: Capture the schematic of your system in Proteus (see Figure 2.1).

Report 2: Implement the function `updateLEDMatrix(int index)`, similarly to the 4-segment-LED case, and update `matrix_buffer` to display the character "A".

Listing 11.1 — Exercise 9 - updateLEDMatrix function

```
void updateLEDMatrix(int index)
```

```

{
    HAL_GPIO_WritePin(ENM0_GPIO_Port, ENM0_Pin, RESET);
    HAL_GPIO_WritePin(ENM1_GPIO_Port, ENM1_Pin, RESET);
    HAL_GPIO_WritePin(ENM2_GPIO_Port, ENM2_Pin, RESET);
    HAL_GPIO_WritePin(ENM3_GPIO_Port, ENM3_Pin, RESET);
    HAL_GPIO_WritePin(ENM4_GPIO_Port, ENM4_Pin, RESET);
    HAL_GPIO_WritePin(ENM5_GPIO_Port, ENM5_Pin, RESET);
    HAL_GPIO_WritePin(ENM6_GPIO_Port, ENM6_Pin, RESET);
    HAL_GPIO_WritePin(ENM7_GPIO_Port, ENM7_Pin, RESET);

    uint8_t data = matrix_buffer[index];
    HAL_GPIO_WritePin(ROW0_GPIO_Port, ROW0_Pin, (data & 0x01) ? SET : RESET);
    HAL_GPIO_WritePin(ROW1_GPIO_Port, ROW1_Pin, (data & 0x02) ? SET : RESET);
    HAL_GPIO_WritePin(ROW2_GPIO_Port, ROW2_Pin, (data & 0x04) ? SET : RESET);
    HAL_GPIO_WritePin(ROW3_GPIO_Port, ROW3_Pin, (data & 0x08) ? SET : RESET);
    HAL_GPIO_WritePin(ROW4_GPIO_Port, ROW4_Pin, (data & 0x10) ? SET : RESET);
    HAL_GPIO_WritePin(ROW5_GPIO_Port, ROW5_Pin, (data & 0x20) ? SET : RESET);
    HAL_GPIO_WritePin(ROW6_GPIO_Port, ROW6_Pin, (data & 0x40) ? SET : RESET);
    HAL_GPIO_WritePin(ROW7_GPIO_Port, ROW7_Pin, (data & 0x80) ? SET : RESET);

    switch (index) {
        case 0: HAL_GPIO_WritePin(ENM0_GPIO_Port, ENM0_Pin, SET); break;
        case 1: HAL_GPIO_WritePin(ENM1_GPIO_Port, ENM1_Pin, SET); break;
        case 2: HAL_GPIO_WritePin(ENM2_GPIO_Port, ENM2_Pin, SET); break;
        case 3: HAL_GPIO_WritePin(ENM3_GPIO_Port, ENM3_Pin, SET); break;
        case 4: HAL_GPIO_WritePin(ENM4_GPIO_Port, ENM4_Pin, SET); break;
        case 5: HAL_GPIO_WritePin(ENM5_GPIO_Port, ENM5_Pin, SET); break;
        case 6: HAL_GPIO_WritePin(ENM6_GPIO_Port, ENM6_Pin, SET); break;
        case 7: HAL_GPIO_WritePin(ENM7_GPIO_Port, ENM7_Pin, SET); break;
        default: break;
    }
}

uint8_t matrix_buffer[8] = {0x00, 0xFC, 0x12, 0x11, 0x11, 0x12, 0xFC, 0x00}; // character
"A"

```

12. Exercise 10: Full Analog Matrix Animation

Report 1: Briefly describe your solution and present your source code.

Solution: add a function `shiftMatrixLeft()` that performs a circular left shift of the `matrix_buffer[8]` array by one position (the first column is saved and moved to the end of the array), producing a left-shifting animation effect on the LED matrix. This function is invoked periodically through a dedicated software timer (timer4, 300ms period) to control the animation speed, independently of timer3 (10ms period) which is used to scan/refresh the matrix columns (a faster scan rate than the 7-segment LEDs, because the matrix has 8 columns to cycle through every refresh, which helps avoid visible flicker).

Listing 12.1 — Exercise 10 - shiftMatrixLeft function

```

// Shift the whole image one column to the left; the first column
// wraps around to become the new last column (Exercise 10)
void shiftMatrixLeft(void)
{
    uint8_t first = matrix_buffer[0];
    for (int i = 0; i < MAX_LED_MATRIX - 1; i++) {
        matrix_buffer[i] = matrix_buffer[i + 1];
    }
    matrix_buffer[MAX_LED_MATRIX - 1] = first;
}

```

The main loop and the interrupt callback are updated to handle all 5 software timers (timer0: clock + DOT, 1000ms; timer1: 7-segment scan, 250ms; timer2: D1 blink, 500ms; timer3: matrix scan, 10ms; timer4: left-shift animation, 300ms):

Listing 12.2 — Exercise 10 - main loop and ISR

```
while (1)
{
    /* USER CODE END WHILE */
    if (timer0_flag == 1) {
        HAL_GPIO_TogglePin(DOT_GPIO_Port, DOT_Pin);
        second++;
        if (second >= 60) { second = 0; minute++; }
        if (minute >= 60) { minute = 0; hour++; }
        if (hour >= 24) { hour = 0; }
        updateClockBuffer();
        setTimer0(1000);
    }

    if (timer1_flag == 1) {
        update7SEG(index_led++);
        if (index_led >= MAX_LED) { index_led = 0; }
        setTimer1(250);
    }

    if (timer2_flag == 1) {
        HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);
        setTimer2(500);
    }

    if (timer3_flag == 1) {
        updateLEDMatrix(index_led_matrix++);
        if (index_led_matrix >= MAX_LED_MATRIX) { index_led_matrix = 0; }
        setTimer3(10);
    }

    if (timer4_flag == 1) {
        shiftMatrixLeft();
        setTimer4(300);
    }
    /* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM2) {
        timer_run();
        timer1_run();
        timer2_run();
        timer3_run();
        timer4_run();
    }
}
```